Universidade Estadual de Mato Grosso do Sul – UEMS
Pró-Reitoria de Pesquisa, Pós-Graduação e Inovação – PROPPI

FORMULÁRIO 10

RELATÓRIO FINAL DE INICIAÇÃO CIENTÍFICA

Edital: UEMS/CNPq nº 22/2025 – PROPPI/UEMS – PIBIC

Acadêmico: Mateus Silva Fontes da Costa

Orientadora: Mercedes Rocío Gonzales Márquez

Título do projeto: Algoritmos de Visão Computacional com Inteligência Artificial

Curso de graduação: Sistemas de Informação

Unidade: Dourados – MS

Área de conhecimento: Ciências Exatas e da Terra / Ciência da Computação / Metodologia e Técnicas da Computação / Processamento Gráfico (Graphics)

1 Avaliação dos objetivos da pesquisa

Na sua avaliação, os objetivos da pesquisa foram atingidos? Justifique em caso de resposta negativa.

(X) SIM ( ) NÃO

2 Alterações no projeto

Houve alguma mudança? Justifique em caso de alteração.

( ) Título ( ) Metodologia ( ) Carga horária ( ) Cronograma (X) Nenhuma

3 RESUMO DO RELATÓRIO

Durante a execução do projeto de Iniciação Científica “Algoritmos de Visão Computacional com Inteligência Artificial”, foram estudadas e aplicadas técnicas de processamento de imagens, aprendizagem de máquina e aprendizagem profunda, com foco no reconhecimento de letras do alfabeto em LIBRAS a partir de imagens. O trabalho incluiu o estudo de binarização, detecção de bordas, representação matricial de imagens e redes neurais, seguido da organização de aproximadamente 39.000 imagens e do desenvolvimento de uma Rede Neural Convolucional. O conjunto de dados foi dividido em subconjuntos de treinamento, validação e teste, e o modelo foi integrado a uma plataforma web em formato de quiz. A aplicação permite que o usuário observe imagens de sinais, informe a letra correspondente e receba um retorno baseado na classificação produzida pelo modelo. Durante os testes, foram identificadas limitações relacionadas a diferenças de iluminação, enquadramento, fundo e posicionamento da mão. Na versão disponibilizada, optou-se por utilizar imagens previamente selecionadas e testadas, enquanto a captura direta pelo usuário permaneceu como possibilidade de desenvolvimento futuro. O projeto permitiu relacionar os fundamentos de visão computacional ao desenvolvimento de uma aplicação interativa voltada à prática do alfabeto em LIBRAS.

Palavras-chave: visão computacional; inteligência artificial; redes neurais convolucionais; LIBRAS; reconhecimento de padrões.

4 INTRODUÇÃO

A Visão Computacional é uma subárea da Inteligência Artificial que busca desenvolver sistemas capazes de processar, analisar e interpretar informações presentes em imagens e outros dados visuais. Por meio de técnicas de processamento de imagens, aprendizagem de máquina e aprendizagem profunda, torna-se possível desenvolver aplicações capazes de identificar características e reconhecer padrões visuais, realizando tarefas que normalmente dependem da percepção humana.

Entre as técnicas utilizadas nesse campo destacam-se as Redes Neurais Convolucionais (CNNs), modelos de aprendizagem profunda especialmente desenvolvidos para trabalhar com imagens. Essas redes utilizam diferentes camadas para identificar características e padrões relevantes nos dados visuais, possibilitando sua utilização em tarefas de classificação e reconhecimento.

Nesse contexto, o presente projeto de Iniciação Científica teve como foco a aplicação dessas tecnologias no reconhecimento de letras do alfabeto em LIBRAS a partir de imagens, buscando relacionar os conhecimentos teóricos de Visão Computacional com o desenvolvimento de uma aplicação prática.

Ao longo do projeto, além do estudo e implementação dos algoritmos, foi desenvolvida uma plataforma de aprendizagem de LIBRAS em formato de quiz, integrando um modelo de Inteligência Artificial responsável pela identificação das letras apresentadas nas imagens. Dessa maneira, o usuário pode testar seus conhecimentos sobre o alfabeto em LIBRAS enquanto a aplicação utiliza a IA como parte do processo de reconhecimento e classificação das imagens.

O desenvolvimento da plataforma permitiu aplicar, de maneira prática, os conhecimentos adquiridos durante a pesquisa, relacionando Visão Computacional, processamento de imagens e Inteligência Artificial a uma aplicação voltada ao aprendizado de LIBRAS.

4.1 Fundamentação teórica

A visão computacional busca extrair informações de imagens e vídeos para identificar padrões, localizar objetos e interpretar cenas. Em aplicações desse tipo, a aquisição, o pré-processamento, a extração de características e a classificação constituem etapas relacionadas.

O processamento digital de imagens e a aprendizagem de máquina oferecem métodos complementares para essas atividades (Szeliski, 2022). Os estudos realizados na iniciação científica constituem a base dos exemplos apresentados nesta seção (Costa, 2025).

4.1.1 Processamento digital e binarização de imagens

Uma imagem digital pode ser representada por uma matriz de pixels. Em escala de cinza, cada posição armazena uma intensidade; em imagens coloridas, múltiplos canais representam as componentes de cor. O pré-processamento prepara esses dados por meio de operações como conversão de cores, redimensionamento e redução de ruído. A extração de características evidencia formas, texturas ou pontos de interesse. Entre os métodos citados no IC está o SIFT, utilizado para descrever características locais; ele não deve ser confundido com um sistema completo de reconhecimento facial.

A binarização converte a imagem em duas classes de intensidade. Na limiarização binária convencional com saída de 8 bits, pixels acima do limiar recebem 255, e os demais recebem 0. Esses valores representam branco e preto, respectivamente. A Figura 1.0 apresenta o exemplo implementado com OpenCV: a imagem é carregada em escala de cinza, processada com cv2.threshold() e exibida para comparação (Costa, 2025).

Figura 1.0 – Código de binarização de imagem com OpenCV

```python
import cv2
import numpy as np

# Carregar a imagem
imagem = cv2.imread('sua_imagem.jpg', cv2.IMREAD_GRAYSCALE)

# Aplicando a binarização com um valor de threshold (valor escolhido manualmente)
_, imagem_binaria = cv2.threshold(imagem, 127, 255, cv2.THRESH_BINARY)

# Mostrar a imagem binarizada
cv2.imshow('Imagem Binarizada', imagem_binaria)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

O limiar também pode ser calculado automaticamente. O método de Otsu procura separar os grupos de intensidade reduzindo a variância dentro das classes. Em imagens com ruído, um filtro de mediana ou outra operação de suavização pode ser aplicado previamente. A adequação da binarização depende da iluminação e do contraste entre objeto e fundo. As

Figuras 1.1 e 1.2 apresentam, respectivamente, a imagem de teste e o resultado da binarização.

Figura 1.1 – Imagem de teste antes da binarização



Fonte: Reproduzida de Costa (2025).

Nota: Imagem de teste Lenna.

Figura 1.2 – Imagem após a binarização



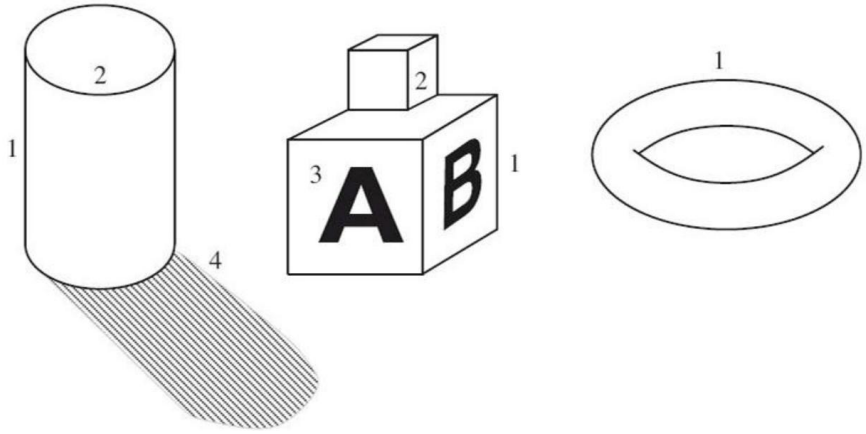
Fonte: Elaboração própria, reproduzida de Costa (2025).

4.1.2 Detecção de bordas

Bordas correspondem a mudanças relevantes de intensidade. Elas podem ser produzidas por descontinuidades de profundidade, alterações na orientação de uma superfície, diferenças de refletância ou mudanças de iluminação. Portanto, uma borda detectada nem

sempre representa o limite entre dois objetos. A Figura 2.0 ilustra situações que produzem essas variações (Russell; Norvig, 2013 apud Costa, 2025).

Figura 2.0 – Descontinuidades associadas à formação de bordas



Fonte: Russell e Norvig (2013 apud Costa, 2025).

No exemplo da Figura 2.1, a imagem é convertida para escala de cinza e suavizada com um filtro gaussiano. O detector Canny é aplicado em seguida para realçar os contornos. A suavização reduz variações pequenas que poderiam interferir na detecção. As Figuras 2.2 e 2.3 apresentam a entrada e o resultado obtido no estudo (Costa, 2025).

Figura 2.1 – Implementação da detecção de bordas com Canny

```python
import cv2  # Importa a biblioteca OpenCV para processamento de imagens
import matplotlib.pyplot as plt  # Importa o matplotlib para exibir imagens

# Lê a imagem do arquivo "imagem.jpg" e armazena na variável img
img = cv2.imread("imagem.jpg")

# Converte a imagem colorida (BGR) para tons de cinza (escala de cinza)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Aplica um filtro Gaussiano para suavizar a imagem e reduzir ruídos
blurred = cv2.GaussianBlur(gray, (3, 3), 0)

# Aplica o detector de arestas Canny na imagem suavizada
# 100 é o limite inferior e 200 é o limite superior para detectar bordas
edges = cv2.Canny(blurred, 100, 200)

# Exibe a imagem resultante usando matplotlib em escala de cinza
plt.imshow(edges, cmap="gray")
plt.title("Arestas detectadas")  # Título da imagem exibida
plt.axis("off")  # Remove os eixos para uma visualização mais limpa
plt.show()  # Mostra a janela com a imagem de arestas detectadas
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

Figura 2.2 – Imagem da mão antes da detecção de bordas



Fonte: Elaboração própria, reproduzida de Costa (2025).

Figura 2.3 – Contornos detectados na imagem da mão



Fonte: Elaboração própria, reproduzida de Costa (2025).

4.1.3 Representação de imagens e aprendizagem de máquina

Na aprendizagem supervisionada, o modelo recebe exemplos associados a rótulos e ajusta seus parâmetros para relacionar os dados às classes desejadas. Em imagens, esses dados podem ser pixels ou características extraídas. A representação matricial facilita a

compreensão de como uma informação visual é convertida em valores numéricos. As Figuras 3.0 a 3.2 apresentam o código, a imagem binária e sua matriz numérica (Costa, 2025).

Figura 3.0 – Código para percorrer e exibir a matriz de uma imagem

```python
from PIL import Image
import numpy as np

# Carregar a imagem
image = Image.open('caminho/para/sua/imagem.png')

# Converter a imagem para escala de cinza (preto e branco)
gray_image = image.convert('L')

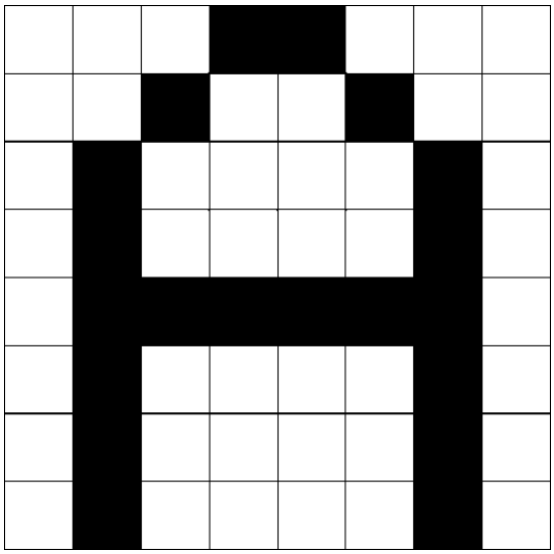
# Converter para numpy array (matriz de pixels)
pixels = np.array(gray_image)

# Normalizar para valores 0 e 1 (preto e branco)
pixels_binarios = (pixels > 128).astype(int)

print(pixels_binarios)
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

Figura 3.1 – Representação visual dos pixels da imagem binária



Fonte: Elaboração própria, reproduzida de Costa (2025).

Figura 3.2 – Valores da matriz da imagem binária

0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	1	1	1	1	1	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0

Fonte: Elaboração própria, reproduzida de Costa (2025).

4.1.4 Redes neurais e perceptron

As redes neurais artificiais são modelos inspirados de maneira abstrata na organização de conexões entre unidades de processamento. A camada de entrada recebe os dados, as camadas ocultas realizam transformações e a camada de saída produz a resposta. Cada unidade combina entradas com pesos e um termo de viés, aplicando em seguida uma função de ativação. O treinamento modifica esses parâmetros para reduzir os erros de previsão (Goodfellow; Bengio; Courville, 2016).

O perceptron é um modelo elementar que permite estudar a soma ponderada, a função de ativação e a atualização de pesos. A Figura 4.0 reproduz o código analisado no IC. O padrão de classes descrito nesse exemplo corresponde ao problema XOR, no qual as entradas [0, 0] e [1, 1] pertencem a uma classe e [0, 1] e [1, 0] pertencem à outra. Um perceptron de camada única não consegue separar corretamente esse padrão; uma rede com camada oculta e ativação não linear pode representar essa relação.

Figura 4.0 – Código didático de um perceptron



```python
import numpy as np

class Perceptron:
    def __init__(self, n_inputs, learning_rate=0.01, n_epochs=100):
        self.weights = np.zeros(n_inputs + 1)  # Pesos e bias
        self.learning_rate = learning_rate
        self.n_epochs = n_epochs

    def activation_function(self, x):
        return 1 if x ≥ 0 else 0  # Função de ativação degrau (limiar em 0)

    def predict(self, inputs):
        summation = np.dot(inputs, self.weights[1:]) + self.weights[0]  # Soma ponderada + bias
        return self.activation_function(summation)

    def train(self, X, y):
        for _ in range(self.n_epochs):
            for inputs, label in zip(X, y):
                prediction = self.predict(inputs)
                error = label - prediction
                self.weights[1:] += self.learning_rate * error * inputs  # Atualiza os pesos
                self.weights[0] += self.learning_rate * error  # Atualiza o bias

# Exemplo de uso do Perceptron
if __name__ == "__main__":
    # Dados de entrada: X = [x1, x2], e y = classe (1 ou 2)
    # Aqui, usamos 0 para Classe 1 e 1 para Classe 2
    X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  # Entradas
    y = np.array([0, 1, 1, 0])  # Classes (0 para Classe 1, 1 para Classe 2)

    perceptron = Perceptron(n_inputs=2)  # Número de entradas = 2
    perceptron.train(X, y)

    # Testando as previsões
    print("Resultados das previsões após treinamento:")
    for inputs in X:
        prediction = perceptron.predict(inputs)
        print(f"Entrada: {inputs}, Predição: {'Classe 1' if prediction == 0 else 'Classe 2'}")

```

Fonte: Elaboração própria, reproduzida de Costa (2025).

Nota: O padrão XOR descrito no estudo não é linearmente separável.

4.1.5 Redes neurais convolucionais e funções de ativação

As redes neurais convolucionais utilizam filtros, ou kernels, que percorrem regiões locais da imagem e produzem mapas de características. Ao longo do treinamento, os filtros são ajustados para reconhecer padrões úteis à tarefa. Camadas iniciais podem responder a bordas e texturas, enquanto camadas posteriores combinam essas informações em representações mais complexas. A etapa final relaciona as características extraídas às classes de interesse (Goodfellow; Bengio; Courville, 2016).

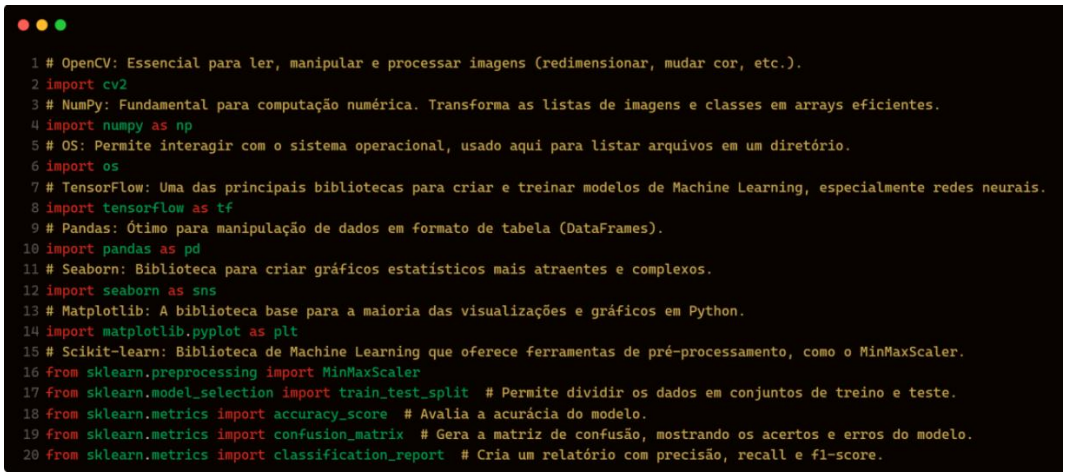
As funções de ativação introduzem não linearidade. A ReLU é utilizada em camadas internas; a sigmoid produz valores entre zero e um e pode ser empregada em uma saída binária; a softmax distribui valores entre classes mutuamente exclusivas. O uso dessas funções depende da tarefa e da arquitetura. As redes densas apresentadas nos exemplos didáticos do IC não devem ser confundidas com a arquitetura convolucional utilizada na aplicação final.

4.1.6 Preparação dos dados e treinamento

O estudo também incluiu uma sequência de implementação para classificação binária. As Figuras 5.0 a 5.5 registram as etapas de importação de bibliotecas, definição de caminhos, leitura das imagens, organização dos dados, construção da rede e compilação. Esses exemplos documentam o aprendizado realizado durante a pesquisa; os parâmetros da implementação final são apresentados na seção 6 (Costa, 2025).

As bibliotecas reúnem as funções de leitura das imagens, manipulação de matrizes, preparação dos dados e construção do modelo.

Figura 5.0 – Importação das bibliotecas para preparação e treinamento

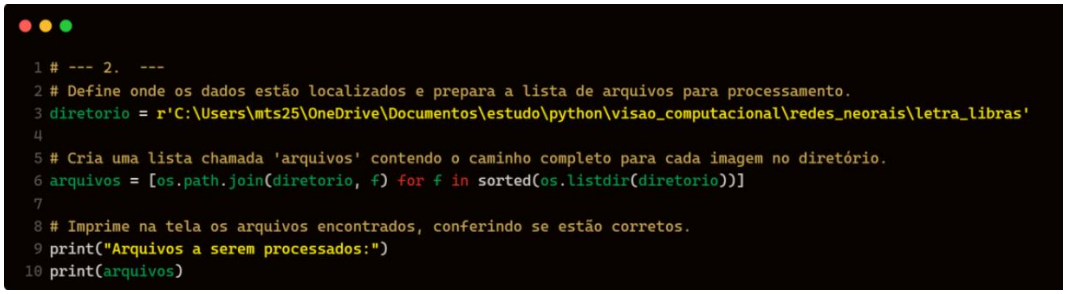


```python
# OpenCV: Essencial para ler, manipular e processar imagens (redimensionar, mudar cor, etc.).
import cv2
# NumPy: Fundamental para computação numérica. Transforma as listas de imagens e classes em arrays eficientes.
import numpy as np
# OS: Permite interagir com o sistema operacional, usado aqui para listar arquivos em um diretório.
import os
# TensorFlow: Uma das principais bibliotecas para criar e treinar modelos de Machine Learning, especialmente redes neurais.
import tensorflow as tf
# Pandas: Ótimo para manipulação de dados em formato de tabela (DataFrames).
import pandas as pd
# Seaborn: Biblioteca para criar gráficos estatísticos mais atraentes e complexos.
import seaborn as sns
# Matplotlib: A biblioteca base para a maioria das visualizações e gráficos em Python.
import matplotlib.pyplot as plt
# Scikit-learn: Biblioteca de Machine Learning que oferece ferramentas de pré-processamento, como o MinMaxScaler.
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split  # Permite dividir os dados em conjuntos de treino e teste.
from sklearn.metrics import accuracy_score  # Avalia a acurácia do modelo.
from sklearn.metrics import confusion_matrix  # Gera a matriz de confusão, mostrando os acertos e erros do modelo.
from sklearn.metrics import classification_report  # Cria um relatório com precisão, recall e f1-score.
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

A organização dos caminhos permite localizar os arquivos e associar os exemplos às classes de interesse.

Figura 5.1 – Configuração dos caminhos das imagens e dos rótulos



```python
# --- 2.  ---
# Define onde os dados estão localizados e prepara a lista de arquivos para processamento.
diretorio = r'C:\Users\mts25\OneDrive\Documentos\estudo\python\visao_computacional\redes_neorais\letra_libras'

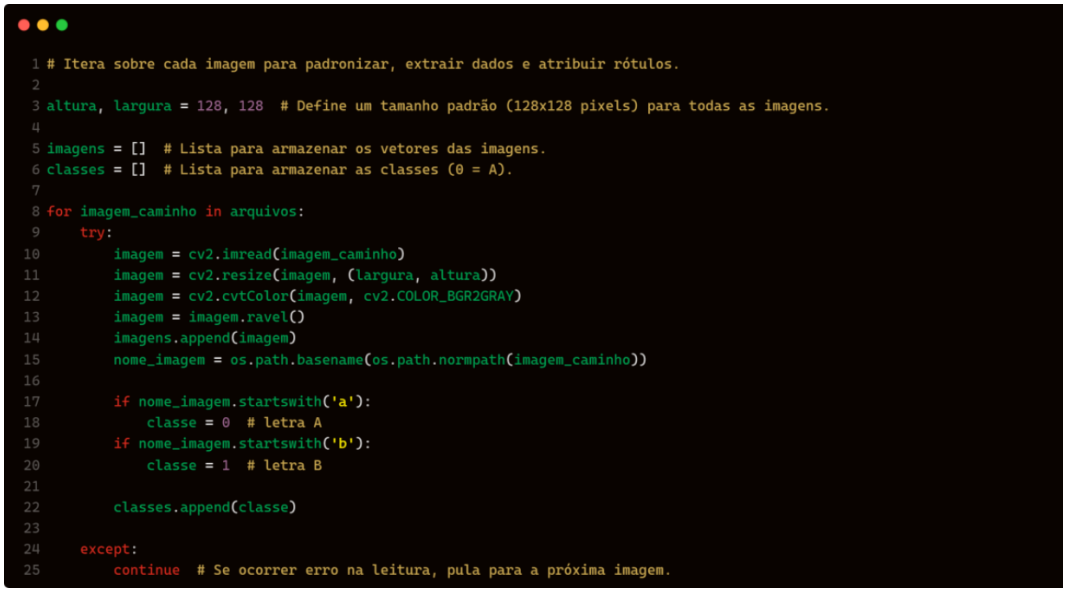
# Cria uma lista chamada 'arquivos' contendo o caminho completo para cada imagem no diretório.
arquivos = [os.path.join(diretorio, f) for f in sorted(os.listdir(diretorio))]

# Imprime na tela os arquivos encontrados, conferindo se estão corretos.
print("Arquivos a serem processados:")
print(arquivos)
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

O procedimento percorre as imagens, prepara suas representações numéricas e registra os rótulos utilizados na aprendizagem supervisionada.

Figura 5.2 – Pré-processamento das imagens e definição das classes



```python
# Itera sobre cada imagem para padronizar, extrair dados e atribuir rótulos.

altura, largura = 128, 128  # Define um tamanho padrão (128x128 pixels) para todas as imagens.

imagens = []  # Lista para armazenar os vetores das imagens.
classes = []  # Lista para armazenar as classes (0 = A).

for imagem_caminho in arquivos:
    try:
        imagem = cv2.imread(imagem_caminho)
        imagem = cv2.resize(imagem, (largura, altura))
        imagem = cv2.cvtColor(imagem, cv2.COLOR_BGR2GRAY)
        imagem = imagem.ravel()
        imagens.append(imagem)
        nome_imagem = os.path.basename(os.path.normpath(imagem_caminho))

        if nome_imagem.startswith('a'):
            classe = 0  # letra A
        if nome_imagem.startswith('b'):
            classe = 1  # letra B

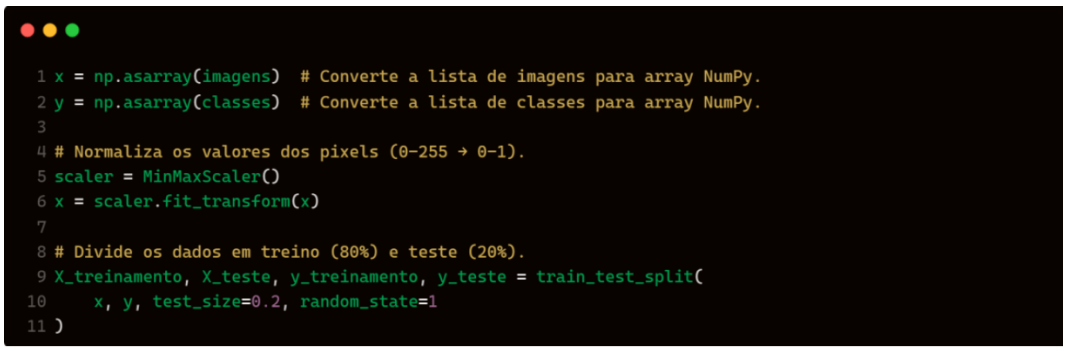
        classes.append(classe)

    except:
        continue  # Se ocorrer erro na leitura, pula para a próxima imagem.
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

No exemplo didático, os dados são convertidos em arrays, normalizados e divididos em 80% para treinamento e 20% para teste. Essa divisão difere da configuração final descrita na metodologia.

Figura 5.3 – Conversão, normalização e divisão dos dados



```python
x = np.asarray(imagens)  # Converte a lista de imagens para array NumPy.
y = np.asarray(classes)  # Converte a lista de classes para array NumPy.

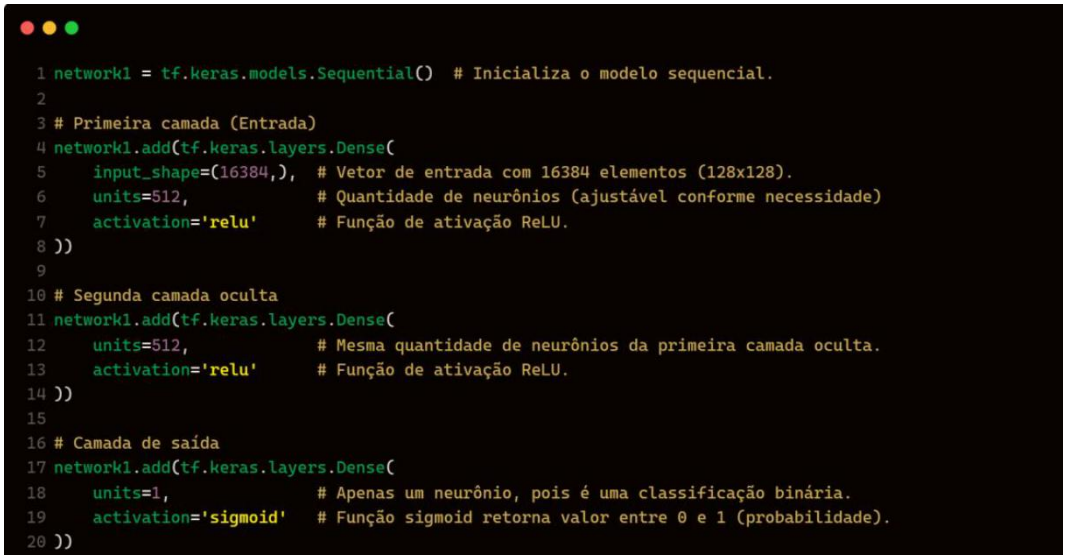
# Normaliza os valores dos pixels (0-255 → 0-1).
scaler = MinMaxScaler()
x = scaler.fit_transform(x)

# Divide os dados em treino (80%) e teste (20%).
X_treinamento, X_teste, y_treinamento, y_teste = train_test_split(
    x, y, test_size=0.2, random_state=1
)
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

A arquitetura organiza camadas de processamento e a saída binária. O número de camadas não equivale ao número total de neurônios da rede.

Figura 5.4 – Definição da arquitetura da rede neural didática



```python
network1 = tf.keras.models.Sequential()  # Inicializa o modelo sequencial.

# Primeira camada (Entrada)
network1.add(tf.keras.layers.Dense(
    input_shape=(16384,),  # Vetor de entrada com 16384 elementos (128x128).
    units=512,             # Quantidade de neurônios (ajustável conforme necessidade)
    activation='relu'      # Função de ativação ReLU.
))

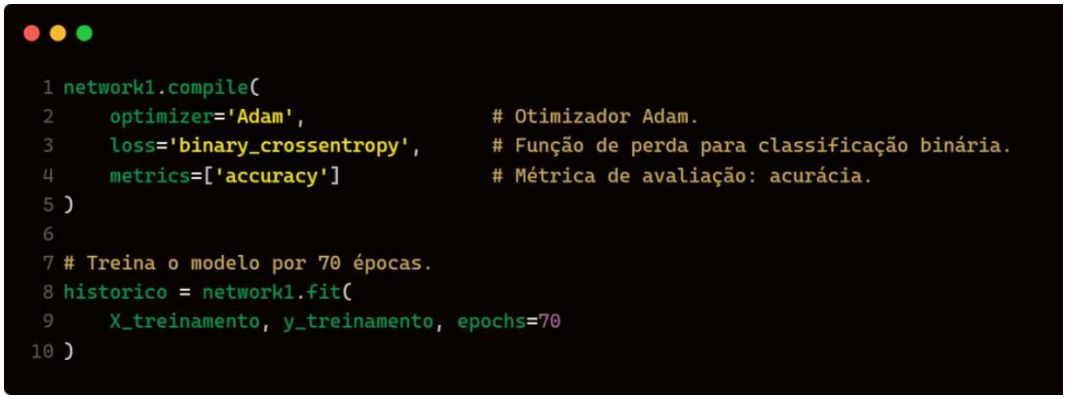
# Segunda camada oculta
network1.add(tf.keras.layers.Dense(
    units=512,             # Mesma quantidade de neurônios da primeira camada oculta.
    activation='relu'      # Função de ativação ReLU.
))

# Camada de saída
network1.add(tf.keras.layers.Dense(
    units=1,               # Apenas um neurônio, pois é uma classificação binária.
    activation='sigmoid'   # Função sigmoid retorna valor entre 0 e 1 (probabilidade).
))
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

O exemplo utiliza Adam, perda binary_crossentropy e acurácia. A perda orienta o ajuste dos parâmetros; a métrica acompanha os acertos.

Figura 5.5 – Compilação e treinamento do modelo didático



```python
network1.compile(
    optimizer='Adam',                # Otimizador Adam.
    loss='binary_crossentropy',      # Função de perda para classificação binária.
    metrics=['accuracy']             # Métrica de avaliação: acurácia.
)

# Treina o modelo por 70 épocas.
historico = network1.fit(
    X_treinamento, y_treinamento, epochs=70
)
```

Fonte: Elaboração própria, reproduzida de Costa (2025).

A avaliação deve utilizar dados separados do treinamento. Quando a normalização exige estimar parâmetros, esses parâmetros devem ser ajustados no conjunto de treinamento e aplicados aos demais conjuntos, evitando que informações de teste influenciem o ajuste. A representatividade das imagens é igualmente relevante: variações de iluminação, fundo e enquadramento precisam ser consideradas para avaliar a aplicação em condições diferentes das utilizadas no desenvolvimento.

5 OBJETIVOS

O projeto teve como objetivo estudar e aplicar técnicas de Visão Computacional e Inteligência Artificial para o reconhecimento de letras do alfabeto em LIBRAS a partir de imagens, conforme definido na proposta inicial.

Durante o desenvolvimento, buscou-se compreender e implementar algoritmos de processamento de imagens, aprendizagem de máquina e aprendizagem profunda, selecionando técnicas adequadas para a identificação e classificação das letras em LIBRAS. Esses objetivos contemplam o estudo dos algoritmos, sua implementação e o desenvolvimento de uma aplicação para detecção das letras.

Além disso, o projeto teve como objetivo integrar o modelo de Inteligência Artificial desenvolvido a uma plataforma de LIBRAS em formato de quiz, permitindo que os usuários testem seus conhecimentos por meio da identificação das letras apresentadas em imagens.

6 METODOLOGIA

O desenvolvimento do projeto foi conduzido em etapas, abrangendo o estudo dos fundamentos de Visão Computacional e Inteligência Artificial, organização e pré-processamento do conjunto de imagens, desenvolvimento e treinamento de uma Rede Neural Convolucional (CNN), avaliação do modelo e integração da Inteligência Artificial a uma plataforma web voltada ao aprendizado de LIBRAS.

6.1 Estruturação e Distribuição do Conjunto de Imagens

Para o desenvolvimento do modelo de Inteligência Artificial, foi utilizado um conjunto de aproximadamente 39.000 imagens, distribuídas entre as diferentes classes correspondentes às letras do alfabeto em LIBRAS utilizadas no projeto. A organização das imagens foi realizada em diretórios separados por classe, permitindo associar automaticamente cada imagem à sua respectiva letra durante o carregamento dos dados.

Após o carregamento, o conjunto de dados foi dividido de maneira estratificada em três subconjuntos: 70% para treinamento, 15% para validação e 15% para teste. A estratificação foi utilizada para preservar a distribuição das diferentes classes em cada subconjunto.

6.2 Pré-processamento das Imagens

Antes do treinamento, as imagens passaram por uma etapa de pré-processamento para padronizar os dados fornecidos à rede neural. Inicialmente, as imagens foram convertidas para o espaço de cores RGB e redimensionadas para 128 × 128 pixels. Em seguida, os valores dos pixels foram convertidos para números de ponto flutuante e normalizados para o intervalo entre 0 e 1.

Também foram utilizadas técnicas de aumento de dados (data augmentation) exclusivamente sobre o conjunto de treinamento. Foram aplicadas variações de rotação, deslocamento horizontal e vertical, zoom, brilho e cisalhamento. Essa estratégia teve como finalidade aumentar a diversidade das amostras apresentadas durante o treinamento e favorecer a capacidade de generalização da rede neural.

6.3 Desenvolvimento da Rede Neural Convolucional

Para realizar a classificação das imagens, foi desenvolvida uma Rede Neural Convolucional (CNN) utilizando TensorFlow/Keras. A arquitetura foi organizada em quatro blocos convolucionais, contendo camadas Conv2D com 32, 64, 128 e 256 filtros. Foram utilizadas ainda camadas de normalização, redução de dimensionalidade por Max Pooling e Dropout.

Após os blocos convolucionais, foi utilizada uma camada GlobalAveragePooling2D, seguida por uma camada totalmente conectada com 256 neurônios e uma camada de saída com função de ativação Softmax, responsável por fornecer as probabilidades correspondentes às diferentes classes de letras.

6.4 Treinamento e Avaliação do Modelo

O modelo foi compilado utilizando o otimizador Adam, com taxa de aprendizado inicial de 0,0001, e a função de perda sparse_categorical_crossentropy. O treinamento foi configurado para até 60 épocas, com lotes de 32 imagens.

Durante o treinamento, foram utilizados mecanismos de Early Stopping, redução automática da taxa de aprendizado e salvamento do modelo que apresentasse o melhor desempenho de validação. Ao final, o modelo foi avaliado utilizando o conjunto de teste, considerando a acurácia, o relatório de classificação e a matriz de confusão.

Uma dificuldade durante o treinamento foi o volume limitado de dados disponíveis para representar a variedade de situações de uso. Embora o conjunto reunisse aproximadamente 39.000 imagens, a quantidade e a diversidade de exemplos para algumas variações de iluminação, fundo, enquadramento e posição da mão foram insuficientes para garantir o mesmo desempenho com imagens capturadas em condições diferentes. Isso exigiu cuidado na avaliação e orientou a seleção das imagens utilizadas na versão atual do quiz.

6.5 Desenvolvimento da Plataforma de LIBRAS

Após o treinamento e a avaliação, o modelo foi integrado a uma plataforma web de aprendizagem de LIBRAS, desenvolvida no formato de quiz. O objetivo da plataforma é possibilitar que o usuário observe imagens representando letras em LIBRAS e teste seus conhecimentos por meio das questões apresentadas.

A aplicação utiliza o modelo treinado para realizar a classificação das imagens, associando as previsões da Inteligência Artificial às letras correspondentes. Dessa maneira, o projeto passou a reunir em uma única aplicação os conhecimentos de Visão Computacional, aprendizagem profunda e desenvolvimento web.

As imagens dos sinais apresentadas no quiz foram geradas com auxílio de inteligência artificial e posteriormente selecionadas e testadas na aplicação. O layout visual da plataforma também foi gerado com auxílio de inteligência artificial e integrado ao desenvolvimento do sistema. Esses recursos compõem a interface; a classificação das letras é realizada pelo modelo treinado no projeto.

7 RESULTADOS E DISCUSSÃO

O desenvolvimento do projeto resultou na construção de um modelo de Inteligência Artificial destinado ao reconhecimento e à classificação de imagens representando letras do alfabeto em LIBRAS. Para o desenvolvimento da Rede Neural Convolucional (CNN), foram utilizadas aproximadamente 39.000 imagens, possibilitando o aprendizado de características visuais presentes nas diferentes classes utilizadas no projeto.

Para avaliar o desempenho do modelo, foi utilizado um conjunto de teste separado das imagens empregadas no treinamento. A avaliação considerou métricas como acurácia, classification report e matriz de confusão, permitindo analisar o desempenho geral do modelo e seu comportamento individual na classificação das diferentes letras.

Além do desenvolvimento e treinamento do modelo, um dos principais resultados obtidos foi sua integração a uma plataforma web voltada ao aprendizado de LIBRAS. A aplicação foi estruturada no formato de quiz, possibilitando que o usuário observe imagens de sinais, informe a letra correspondente e receba um retorno sobre sua resposta.

Dessa forma, o projeto não ficou restrito ao treinamento experimental de uma rede neural, mas resultou em uma aplicação funcional que integra conceitos de Visão Computacional, Inteligência Artificial e desenvolvimento web.

7.1 Plataforma de Aprendizagem de LIBRAS

Como resultado prático do projeto, foi desenvolvida uma plataforma web com o objetivo de proporcionar uma experiência interativa para a prática do alfabeto em LIBRAS. A página inicial apresenta a proposta da aplicação, informações sobre seu funcionamento e acesso ao quiz.

Figura 6.0 – Página inicial da plataforma de aprendizagem de LIBRAS



Fonte: Elaboração própria.

A Figura 6.0 apresenta a página inicial da plataforma, desenvolvida para fornecer ao usuário acesso ao quiz e apresentar a proposta da aplicação.

7.2 Interface do Quiz de LIBRAS

Após acessar o quiz, o usuário é direcionado para a interface responsável pela atividade de identificação das letras. Nessa etapa, são apresentadas imagens contendo diferentes sinais do alfabeto em LIBRAS, permitindo que uma delas seja selecionada para realização do teste.

Figura 6.1 – Interface do quiz de LIBRAS



Fonte: Elaboração própria.

Conforme apresentado na Figura 6.1, a interface do quiz disponibiliza as imagens utilizadas na atividade e um campo destinado à resposta do usuário. Após selecionar uma imagem, o usuário deve observar o sinal apresentado e informar qual letra acredita estar sendo representada.

A plataforma integra o modelo de Inteligência Artificial desenvolvido durante o projeto, responsável pela classificação da imagem selecionada. Essa integração permite utilizar, em uma aplicação prática, o modelo treinado durante as etapas anteriores da pesquisa.

7.3 Identificação de Resposta Incorreta

A plataforma também foi desenvolvida para fornecer um retorno ao usuário após cada tentativa. Dessa maneira, é possível verificar imediatamente se a resposta informada corresponde à classificação realizada pelo modelo de Inteligência Artificial.

Figura 6.2 – Retorno de uma resposta incorreta no quiz de LIBRAS



Fonte: Elaboração própria.

A Figura 6.2 apresenta uma situação em que o usuário informa uma letra diferente daquela identificada pelo modelo. Após a resposta ser enviada, o sistema indica que a tentativa está incorreta e apresenta a letra correspondente à classificação realizada pela Inteligência Artificial.

Esse mecanismo de retorno permite que o usuário identifique seu erro e conheça a resposta esperada antes de continuar a atividade. Dessa forma, a plataforma utiliza o resultado da classificação não apenas para verificar a resposta, mas também como parte da experiência de prática proposta pelo quiz.

7.4 Identificação de Resposta Correta

Além da identificação de respostas incorretas, a aplicação também informa ao usuário quando a letra digitada corresponde à classificação realizada pelo modelo.

Figura 6.3 – Retorno de uma resposta correta no quiz de LIBRAS



Fonte: Elaboração própria.

A Figura 6.3 apresenta uma situação em que a resposta informada pelo usuário corresponde à letra identificada pelo modelo de Inteligência Artificial. Nesse caso, a plataforma informa que a resposta está correta, fornecendo um retorno imediato e permitindo que o usuário prossiga para uma nova tentativa.

Na versão atual da plataforma, são utilizadas imagens previamente selecionadas para a realização do quiz. Essa abordagem foi adotada a partir dos testes realizados durante o desenvolvimento, nos quais foi observado que o modelo pode apresentar variações na classificação dependendo das condições da imagem, principalmente em relação à iluminação. Alterações na luminosidade podem modificar características visuais importantes para o processo de reconhecimento e, consequentemente, interferir na classificação realizada pela Rede Neural Convolucional.

Diante dessa limitação, optou-se por disponibilizar ao usuário um conjunto de imagens previamente selecionadas e adequadas ao reconhecimento pelo modelo. Dessa forma, é possível proporcionar maior consistência durante a utilização da plataforma e permitir que o usuário se concentre na identificação das letras apresentadas no quiz.

Os resultados obtidos também indicam uma possibilidade de aprimoramento futuro do projeto. A ampliação do conjunto de dados com imagens capturadas sob diferentes condições de iluminação, posicionamento, enquadramento e fundo poderá contribuir para aumentar a

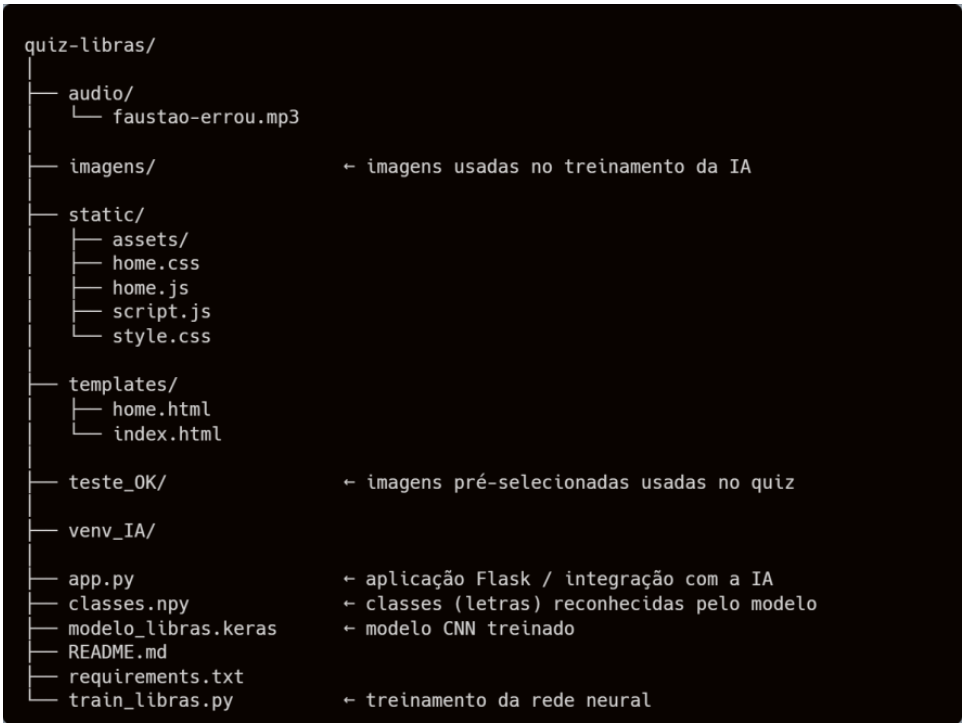
capacidade de generalização do modelo. Com esses aprimoramentos, futuramente poderá ser possível permitir que o próprio usuário forneça ou capture imagens para serem classificadas diretamente pela plataforma.

As imagens das letras utilizadas no quiz foram geradas com o auxílio de Inteligência Artificial e selecionadas para representar os sinais do alfabeto em LIBRAS. Posteriormente, elas foram testadas na plataforma para verificar sua adequação ao processo de classificação realizado pelo modelo desenvolvido.

7.5 Estrutura da Aplicação Desenvolvida

A plataforma foi organizada em uma estrutura de diretórios responsável por separar os diferentes componentes utilizados no desenvolvimento, incluindo o conjunto de imagens, arquivos da interface web, modelo de Inteligência Artificial e código responsável pelo treinamento e execução da aplicação.

Figura 6.4 – Estrutura de arquivos e diretórios da plataforma Quiz Libras



Fonte: Elaboração própria.

Conforme apresentado na Figura 6.4, o diretório imagens contém o conjunto de imagens utilizado durante o processo de treinamento da Rede Neural Convolucional. Já o diretório teste_OK armazena as imagens previamente selecionadas e testadas que são disponibilizadas ao usuário durante a utilização do quiz.

O arquivo train_libras.py contém o código utilizado para o treinamento e avaliação da Rede Neural Convolucional. Após o treinamento, o modelo é armazenado no arquivo modelo_libras.keras, enquanto o arquivo classes.npy mantém as informações referentes às classes utilizadas na classificação das letras.

A integração entre o modelo de Inteligência Artificial e a plataforma web é realizada pelo arquivo app.py, responsável pela execução da aplicação e pelo processamento das solicitações realizadas pelo usuário. A interface da plataforma está organizada nos diretórios templates e static, que armazenam, respectivamente, as páginas HTML e os arquivos de estilização e comportamento da interface.

Essa organização permitiu separar os componentes responsáveis pelo treinamento da Inteligência Artificial, armazenamento das imagens, execução do modelo e interface da aplicação, facilitando o desenvolvimento, os testes e a integração entre o modelo treinado e o quiz.

7.6 Limitações Identificadas e Disponibilização do Projeto

Inicialmente, previa-se que a plataforma permitisse ao usuário capturar uma imagem da própria mão para realizar o reconhecimento automático dos sinais em LIBRAS. Para isso, foram realizados testes com imagens obtidas por câmera e com técnicas de detecção e recorte da mão antes do envio da imagem ao modelo de classificação.

Entretanto, verificou-se que o desempenho do modelo variava quando as imagens capturadas apresentavam condições diferentes das imagens utilizadas durante o treinamento. Fatores como iluminação, sombras, qualidade da câmera, distância, ângulo da mão, enquadramento e fundo interferiram na classificação, podendo ocasionar resultados incorretos ou pouco consistentes. O recorte automático da mão também nem sempre mantinha o mesmo padrão de centralização e proporção das imagens do conjunto de dados.

Além das diferenças visuais entre as imagens, a limitação do volume de dados de treinamento para contemplar essas condições dificultou a generalização do modelo. A ampliação e diversificação do conjunto de imagens constitui, portanto, uma etapa necessária para aperfeiçoar a classificação de capturas feitas diretamente pelo usuário.

Dessa forma, na versão atual, optou-se pela utilização de imagens previamente selecionadas e testadas, proporcionando maior padronização das entradas e maior consistência durante a realização do quiz. A possibilidade de captura direta pelo usuário permanece como

uma melhoria futura, juntamente com a ampliação do conjunto de dados e o aperfeiçoamento do préprocessamento das imagens.

Como resultado do desenvolvimento, a plataforma foi disponibilizada on-line para acesso e utilização do quiz de LIBRAS. A aplicação pode ser acessada pelo link: https://quiz-libras.onrender.com/quiz.

O código-fonte do projeto, bem como os arquivos utilizados no desenvolvimento da aplicação, estão disponíveis no repositório GitHub: https://github.com/mateus-fontes-tech/Quiz-Libras-A. O repositório também contém a documentação do código e informações necessárias para compreender a estrutura e a execução da plataforma.

8 CONSIDERAÇÕES FINAIS

O desenvolvimento deste projeto possibilitou o estudo e a aplicação prática de conceitos relacionados à Visão Computacional, processamento de imagens, Inteligência Artificial e aprendizagem profunda, tendo como foco o reconhecimento e a classificação de letras do alfabeto em LIBRAS.

Ao longo da pesquisa, foi desenvolvida e treinada uma Rede Neural Convolucional (CNN) utilizando aproximadamente 39.000 imagens. O processo envolveu a organização e o préprocessamento das imagens, a divisão do conjunto de dados, o treinamento da rede e a avaliação do modelo, permitindo aplicar os conhecimentos estudados durante a Iniciação Científica em um problema prático.

Como resultado, o modelo treinado foi integrado a uma plataforma web de aprendizagem de LIBRAS, desenvolvida no formato de quiz. A aplicação disponibiliza imagens previamente selecionadas para que o usuário identifique a letra representada e informe sua resposta. Em seguida, o sistema utiliza a classificação realizada pelo modelo de Inteligência Artificial para verificar a resposta e informar ao usuário se houve acerto ou erro.

Durante o desenvolvimento, também foram identificadas limitações no modelo, principalmente relacionadas às variações de iluminação das imagens, que podem interferir no processo de classificação. Diante dessa limitação, optou-se pela utilização de um conjunto de imagens previamente selecionadas e testadas na versão atual da plataforma, proporcionando maior consistência durante a execução do quiz.

Dessa forma, considera-se que os objetivos estabelecidos para o projeto foram alcançados, uma vez que foram estudadas e implementadas técnicas de Visão Computacional e aprendizagem profunda, desenvolvido e treinado um modelo para classificação das letras em LIBRAS e realizada sua integração a uma aplicação web funcional.

Como trabalhos futuros, pretende-se ampliar e diversificar o conjunto de dados, incluindo imagens obtidas em diferentes condições de iluminação, posicionamento das mãos, enquadramento e fundo. Com essas melhorias, espera-se aumentar a capacidade de generalização e a robustez do modelo, possibilitando futuramente que o próprio usuário forneça ou capture imagens para classificação diretamente pela plataforma.

REFERÊNCIAS

BRADSKI, Gary. **The OpenCV Library**. Dr. Dobb's Journal of Software Tools, 2000.

COSTA, Mateus Silva Fontes da. **Algoritmos de visão computacional com inteligência artificial**. Dourados, 2025. Material de pesquisa não publicado. Documento fornecido pelo autor (IC.pdf).

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep learning**. Cambridge, MA: MIT Press, 2016. Disponível em: https://www.deeplearningbook.org/. Acesso em: 28 set. 2026.

PERDIGOTO, Bruno Alves; NAVARRO, Ébervan; PASSERINI, Jefferson Antonio Ribeiro. **Visão computacional: abordagem prática de modelo para construção de detector de linguagem Libras utilizando IA**. Jales: Faculdade de Tecnologia Prof. José Camargo, 2025. Artigo científico apresentado na VII Jornada Acadêmica, Científica e Tecnológica. Disponível em: https://ric.cps.sp.gov.br/handle/123456789/39795. Acesso em: 28 set. 2026.

SZELISKI, Richard. **Computer vision: algorithms and applications**. 2. ed. Cham: Springer, 2022. DOI: 10.1007/978-3-030-34372-9.

DECLARAÇÃO

Por ser a expressão da verdade, firmo a presente declaração ficando responsável pela veracidade das informações contidas neste relatório e ciência do conteúdo da Resolução CEPE-UEMS nº 2.771, de 23 de fevereiro de 2024.

Dourados/MS, 28 de setembro de 2026

_____

Assinatura do bolsista

_____

Assinatura da orientadora

OBS.: Enviar somente a via digital para o e-mail iniciacaocientifica@uems.br.